

public access

WHITE PAPER

SPT Framework for TwinCAT 3

A Practical Guide for Industrial Software Engineers

From ISA-88 Standards to Production-Ready SPT Code

Author

Emad Eddin Abdelghany

TwinCAT® / CEM® / CEA®

Industrial Automation & Energy

Series Status

Articles 1–7 of 12 — To be continued

Version 1.0 — 2025

Free distribution

Foreword

[public access](#)

Industrial software engineers working with Beckhoff TwinCAT 3 frequently encounter the SPT (Structured PackML Template) Framework as a recommended architecture for machine control programs. While the framework is powerful and well-designed, the entry point can feel steep: the documentation assumes familiarity with its layered structure, and the connection between the familiar PackML standard and the new SPT abstractions is not always immediately obvious.

This white paper bridges that gap. It is written for the practicing industrial automation engineer who already understands PackML and ISA-88 — the standards on which SPT is built — and who needs a structured, code-level explanation of how to apply the SPT framework in real TwinCAT 3 projects. No background in object-oriented programming theory is assumed; every concept is introduced through code examples drawn from a consistent machine scenario (a filling station) that grows in complexity across articles.

This paper is entirely free and addressed to whom it may concern. It may be shared, printed, or referenced without restriction. The only goal is to reduce the time it takes for a capable automation engineer to become productive with SPT.

The series currently covers Articles 1 through 7. Additional articles covering the Control Source, Alarms and Events, Machine Code Progression, and a complete end-to-end SPT implementation will follow in subsequent releases.

Prerequisites

The reader is expected to have working knowledge of:

- PackML state machine concepts (STOPPED, IDLE, EXECUTE, and the 17 PackML states)
- ISA-88 physical model (Equipment Module, Control Module hierarchy)
- TwinCAT 3 PLC programming environment and basic Structured Text (ST)
- Basic understanding of function blocks (FBs) in IEC 61131-3

Table of Contents public access

Foreword	2
Table of Contents	3
Chapter 1: ISA-88 and the Physical Model	6
1.1 What ISA-88 Is	6
1.2 The Physical Model.....	6
1.3 Why the Physical Model Matters for Software	6
Chapter 2: PackML — The State Machine Standard	8
2.1 What PackML Is	8
2.2 The 17 States	8
2.3 Acting vs. Wait States	8
2.4 Operator Commands and Valid Transitions	9
Chapter 3: The SPT Framework — Articles 1 to 7	10
The Problem Before SPT	10
What SPT Actually Is	10
The Core Idea in One Sentence	10
The Four Libraries	11
SPT vs. Raw PackML — Side by Side	11
What SPT Does Not Do	11
Version Note.....	11
The Physical World Maps to Three Levels.....	13
Level 1 — The Component	13
Level 2 — The Equipment Module (Submodule)	13
Level 3 — The Machine Module.....	14
How a Command Flows — Top Down	14
How Completion Propagates — Bottom Up	15
Does Every Component Need an Equipment Module?.....	15
The Hierarchy Decision Rule.....	15
What This Article Covers	17
The Dependency Chain.....	17
Library 1 — SPT_Base_Types.....	17
Function Blocks in SPT_Base_Types	17
What FB_BaseFB Gives Every FB	18
What FB_ComponentBase Adds.....	18
Interfaces in SPT_Base_Types	18
Data Types in SPT_Base_Types	19

Library 2 — SPT_Components	19
What is Pre-Built	19
The Key Rule for SPT_Components	19
Library 3 — SPT_Uilities	19
What Is Inside	19
Library 4 — SPT_Event_Logger	20
What It Provides	20
How They Map to the Hierarchy	20
Reference Addition Order in TwinCAT	21
Where This Fits in the Thread	22
The Inheritance Chain	22
The Full Method Map	22
The Entry Point: CyclicLogic()	23
Lifecycle Method 1: Initialize()	23
Lifecycle Method 2: CyclicLogic() — The Override	24
The State Methods	25
The Naming Convention	25
The SUPER^ Call Rule	25
StateComplete() — The Transition Signal	26
Declaring Your Modules — The First Code	26
Equipment Module	26
Machine Module	26
The Lifecycle Sequence — From Power-On to Execute	27
What the Framework Hides	27
Where This Fits in the Thread	29
The Inheritance Chain	29
The Method Map	29
The Internal Status Model	30
Initialize() — One-Time Setup	30
CyclicLogic() — FINAL, Not Overridable the Same Way	31
HMICommunication() — Component-Level HMI Commands	32
Monitoring(), CreateEvents(), and Alarms	32
Complete Worked Example — FB_DosingValve	33
How the Equipment Module Uses This Component	35
Why This Makes the Component Portable	35
Summary Table — Module vs. Component	36
Where This Fits in the Thread	37

The Core Concept public access	37
What Registration Does Internally	37
V4 Registration Pattern	38
Registration in the Machine Module.....	38
Registration in the Equipment Module	38
The Propagation Path After Registration	39
The Name Property — Set During Registration.....	40
What Happens If You Forget to Register	40
Complete Registration Pattern — Rule Summary	40
Where This Fits in the Thread	42
The Internal _InitComplete Gate	42
How Many Scans Does Initialize() Take?	42
What to Put Inside Initialize() — and What Not To	43
Always goes in Initialize()	43
Never goes in Initialize()	43
I/O Linking Patterns	43
Pattern 1 — REFERENCE TO via a setter method	43
Pattern 2 — ADR() with POINTER TO	44
CyclicLogic() — The Execution Order Contract.....	44
Breaking the Order — What Goes Wrong.....	45
MAIN — The Entry Point Options.....	46
Option A — Simple (recommended for most projects)	46
Option B — MAIN with its own Initialize()	46
The Startup Sequence — First Five Scans.....	47
To Be Continued	50

Chapter 1: ISA-88 and the Physical Model

1.1 What ISA-88 Is

ISA-88 (also known as ANSI/ISA-88 or S88) is a standard for batch process control developed by the International Society of Automation. Although it originated in batch chemical processing, its physical and procedural models have been adopted far beyond that domain — including discrete manufacturing, packaging, and any machine where a defined sequence of states must be executed to produce a result.

The standard defines two things that directly affect how machine control programs are structured: a Physical Model and a Procedural Model. This paper focuses on the Physical Model, because SPT is a direct implementation of it.

1.2 The Physical Model

The ISA-88 Physical Model defines a hierarchy of physical entities that make up a manufacturing system, from the highest level (the enterprise) down to the lowest (control modules). For machine-level automation, the relevant levels are:

Level	ISA-88 Term	Responsibility
3	Unit	Performs a defined set of process actions — the complete machine
2	Equipment Module	A functional grouping of equipment that performs a specific minor activity
1	Control Module	The lowest-level addressable equipment grouping — sensors, actuators, drives

These three levels map directly onto SPT’s Machine Module, Equipment Module, and Component hierarchy, which is detailed in Article 2.

1.3 Why the Physical Model Matters for Software

The Physical Model is not just an organizational diagram. It is a design contract: each level has a defined scope of responsibility, and crossing those boundaries creates software that is hard to maintain and impossible to reuse. Specifically:

- A Control Module (SPT Component) owns exactly one device. It does not make decisions about what the machine is doing.
- An Equipment Module (SPT Equipment Module) coordinates its Control Modules to perform one functional activity. It does not touch I/O directly.
- A Unit (SPT Machine Module) manages the overall machine state and coordinates its Equipment Modules. It does not know how individual devices work.

SPT enforces these responsibilities through its class hierarchy, method names, and registration system.

Understanding the ISA-88 intent behind each level is what makes SPT design decisions feel logical rather than arbitrary.

Chapter 2: PackML — The State Machine Standard

2.1 What PackML Is

PackML (Packaging Machine Language) is a machine state model standardized by OMAC (Organization for Machine Automation and Control) and incorporated into ISA-TR88.00.02. It defines 17 machine states, the valid transitions between them, and the commands that trigger those transitions. In TwinCAT 3, PackML is implemented in the Tc3_PackML_V2 library, which SPT wraps and extends.

2.2 The 17 States

Category	State	Type	Description
Production	STOPPED	Wait	Machine at rest. Only Reset is valid.
Production	RESETTING	Acting	Preparing machine for production — homing, initializing.
Production	IDLE	Wait	Ready for production. Awaiting Start command.
Production	STARTING	Acting	Verifying preconditions before Execute.
Production	EXECUTE	Wait	Active production. Runs until Stop, Hold, or Suspend.
Production	COMPLETING	Acting	End-of-batch wrap-up.
Production	COMPLETE	Wait	Batch finished. Awaiting Reset.
Hold	HOLDING	Acting	Safely pausing from Execute — reach a defined hold position.
Hold	HELD	Wait	Machine paused. Awaiting UnHold.
Hold	UNHOLDING	Acting	Resuming from Held — re-verify before Execute.
Suspend	SUSPENDING	Acting	External condition pausing production.
Suspend	SUSPENDED	Wait	Waiting for external condition to clear.
Suspend	UNSUSPENDING	Acting	Resuming from Suspended.
Stop	STOPPING	Acting	Controlled stop from any running state.
Abort	ABORTING	Acting	Emergency shutdown — no safe sequence required.
Abort	ABORTED	Wait	Emergency stop state. Only Clear is valid.
Abort	CLEARING	Acting	Recovery after Abort. Leads to STOPPED.

2.3 Acting vs. Wait States

The distinction between Acting and Wait states is fundamental. Acting states are transient — the machine is doing something (homing, verifying, stopping) and the state machine waits for a StateComplete signal before advancing. Wait states are stable — the machine remains in them until an operator command or external event triggers a transition.

In raw PackML (without SPT), you process the StateComplete signal manually in your MAIN program. In SPT, you call THIS^.StateComplete() from inside the state method itself — the framework handles the transition. This difference is central to why SPT reduces boilerplate and prevents wiring mistakes.

2.4 Operator Commands and Valid Transitions

Command	Valid From	Goes To
Reset	STOPPED, COMPLETE	RESETTING
Start	IDLE	STARTING
Stop	IDLE, EXECUTE, HELD, SUSPENDED, COMPLETING, COMPLETE	STOPPING
Hold	EXECUTE	HOLDING
UnHold	HELD	UNHOLDING
Suspend	EXECUTE	SUSPENDING
UnSuspend	SUSPENDED	UNSUSPENDING
Abort	ANY state	ABORTING
Clear	ABORTED	CLEARING

These commands, valid states, and transition rules are enforced automatically by PS_PackML_StateMachine_Auto (the underlying PackML FB inside SPT). You do not implement transition guards yourself — the framework rejects invalid commands silently.

Chapter 3: The SPT Framework — Articles 1 to 7

The following seven articles build a complete picture of the SPT framework from first principles, presented in full exactly as developed. Each article is self-contained and builds directly on the previous. All code examples use the Filling Station machine scenario established in the PackML session that preceded this framework series.

Art. 1

What SPT Is and Why It Exists

The Problem Before SPT

When you write PackML without any framework you end up with a flat structure: one MAIN program, one state machine FB, and a collection of individual state FBs wired together manually. This works for small machines, but as soon as the machine grows — two axes, a conveyor, a vision system, a dosing valve — you face three recurring problems:

- Problem 1 — No ownership. Who is responsible for the conveyor during Resetting? Is it the axis FB? The MAIN program? There is no defined answer, so each programmer decides differently. The result is scattered logic that is hard to read and impossible to reuse.
- Problem 2 — No isolation. Your Execute step machine, your alarm logic, and your HMI communication all live in the same method. Fixing a fault condition risks breaking the production sequence.
- Problem 3 — No reuse. The cylinder FB you wrote for this project cannot be dropped into the next project without adaptation, because it is entangled with this machine's state machine.

SPT solves all three by imposing a structure on top of PackML.

What SPT Actually Is

SPT is a collection of libraries that facilitates the creation of standardized machine code and can be reused on future projects due to its modularity and flexibility.

It does not replace PackML. It is built on top of Tc3_PackML_V2. The state machine — STOPPED, IDLE, EXECUTE, and all 17 states — still runs underneath. What SPT adds is:

- A defined hierarchy for organizing your machine into levels
- A set of base classes that hide the raw PackML wiring
- A method-per-state pattern so your code goes in the right place automatically
- A registration system that propagates state transitions through the whole hierarchy automatically
- A component model for reusable, portable hardware control blocks

The Core Idea in One Sentence

Core idea

In SPT, you stop writing against the state machine and start writing inside it — your code lives in methods named after states, and the framework calls them at the right time. In raw PackML you manually wired

StateComplete in MAIN. In SPT, you call THIS^.StateComplete() from inside the state method itself. The framework handles the rest.

The Four Libraries

The four SPT libraries are SPT_Base_Types, SPT_Components, SPT_Event_Logger, and SPT_Uilities.

Library	Role	What you use from it
SPT_Base_Types	Core framework	FB_PackML_BaseModule, FB_ComponentBase, all interfaces
SPT_Components	Pre-built hardware blocks	Ready cylinders, sensors, motion components
SPT_Uilities	Small reusable helpers	Analog scaling, data buffers, comparators, timers
SPT_Event_Logger	Alarm/event system	Wrapper around Tc3_EventLogger tuned for SPT

You always need SPT_Base_Types. The others you add as your machine needs them.

SPT vs. Raw PackML — Side by Side

Aspect	Raw PackML	SPT
State machine	You call the FB in MAIN, wire StateComplete manually	Base class handles it; you override state methods
Output gating	You write IF ST_Execute AND fbMyFB.done in MAIN	Framework calls your M_Execute() only when in Execute
Multi-level machines	No structure — everything in MAIN	Machine Module → Equipment Module → Component hierarchy
Component reuse	FB is tightly coupled to the project	Component inherits FB_ComponentBase → portable
Alarm handling	Ad-hoc per project	Structured FB_Alarm per component, monitored by parent module

What SPT Does Not Do

SPT does not generate code. It does not configure your hardware. It does not replace TwinCAT's axis libraries or I/O mapping. It is purely an application-layer framework — an agreed set of base classes and patterns that sits between your machine logic and the PackML state machine.

Version Note

With the release of PackML V3 for TwinCAT 3.1.4026, the SPT Libraries have moved to V4.0 to support the new PackML standard. The articles in this series use V4. If you are on an older TwinCAT build, SPT V3 uses Tc3_PackML (not _V2) and the base class names are slightly different, but the concepts are identical.

[public access](#)

Art. 2 The Three-Level Hierarchy

The Physical World Maps to Three Levels

Every machine can be decomposed physically. A filling station has a frame, a servo axis, a dosing valve, and sensors. A larger machine has multiple such stations. ISA-88 formalizes this decomposition into levels, and SPT implements it directly in code.

The three levels are:

```
Machine Module (MM)
├── Equipment Module / Submodule (EM)
│   └── Component
```

Each level has a specific responsibility. Crossing responsibilities is the most common mistake beginners make.

Level 1 — The Component

Components are used to control hardware such as an axis, a cylinder, or any other device connected to the I/O. They can also be the connection point to a database, implement a client/server communication architecture, or add functionality to other components.

The Component is the lowest level. It owns exactly one physical device or one logical concern. Examples:

Component	What it owns
FB_DosingValve	One digital output — open/close
FB_NozzleAxis	One servo axis — move/home/status
FB_BottleSensor	One digital input — present/absent
FB_FillConveyor	One conveyor motor

The Component's responsibility is narrow and fixed: drive its device, report its status, raise its own alarms. It does not know what state the machine is in. It does not decide when to run. It only responds to commands given to it by its parent module.

Portability rule

Components can work without the framework and therefore can be developed independently and tested outside of the framework. This is a key design rule — if your component cannot run standalone, it is too tightly coupled.

Level 2 — The Equipment Module (Submodule)

The Equipment Module represents one functional section of the machine. It groups the components that belong together and coordinates them through the PackML states.

For the filling station: **public access**

```
FB_FillingStation ← Equipment Module
├── FB_NozzleAxis      (Component)
├── FB_DosingValve     (Component)
└── FB_BottleSensor   (Component)
```

The EM's responsibility: know what its components should do in each state, issue commands to them, and report completion upward when all its components have finished their state work.

The EM has state methods — M_Resetting, M_Execute, M_Stopping, etc. — where it issues commands to its components. It does not touch physical I/O directly. All I/O goes through its components.

The PackML standard defines only Machine, Equipment, and Component. However there has occasionally been a need to organize machines with multiple levels of Equipment Modules. To accommodate this the SPT Framework uses the term Submodule instead of Equipment Module. This means an EM can itself contain other EMs (called Submodules), giving you a third nesting level when genuinely needed. The official guidance is clear though: more than three levels total is strongly discouraged.

Level 3 — The Machine Module

The Machine Module is the single top-level FB in every SPT project. There is exactly one per machine.

```
FB_FillingMachine ← Machine Module
├── FB_FillingStation (Equipment Module)
│   ├── FB_NozzleAxis
│   ├── FB_DosingValve
│   └── FB_BottleSensor
```

Each project will hold a single Machine Module that is at the top of the hierarchy and contains the Equipment Modules which represent the parts of the machine. This Machine Module (MM) will be responsible for controlling and monitoring each of the Equipment Modules (EMs).

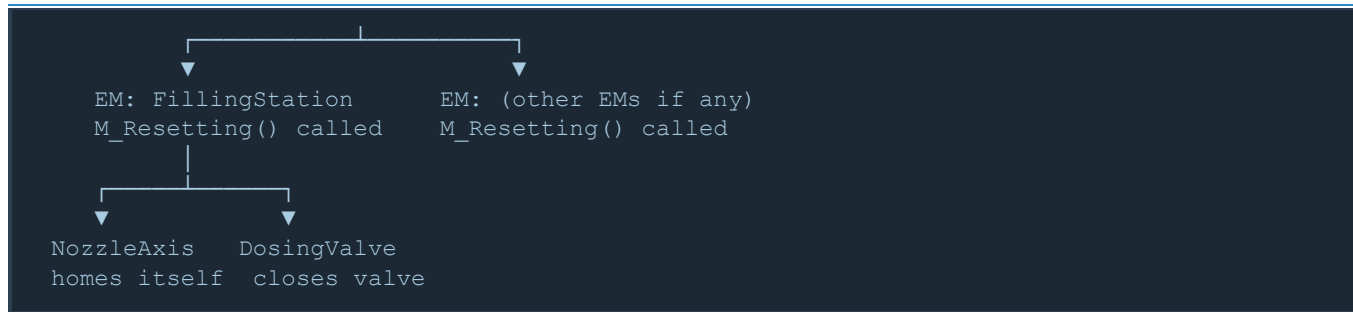
The MM's responsibility: own the PackML state machine, receive operator commands (Start, Stop, Reset...), propagate state transitions downward to all EMs, and aggregate their completions before advancing the state.

The MM also owns the Control Source — the bridge to the HMI. That topic belongs to Article 9, but the key point here is that operator commands arrive at the MM first, never directly at an EM or Component.

How a Command Flows — Top Down

When the operator presses Reset on the HMI:

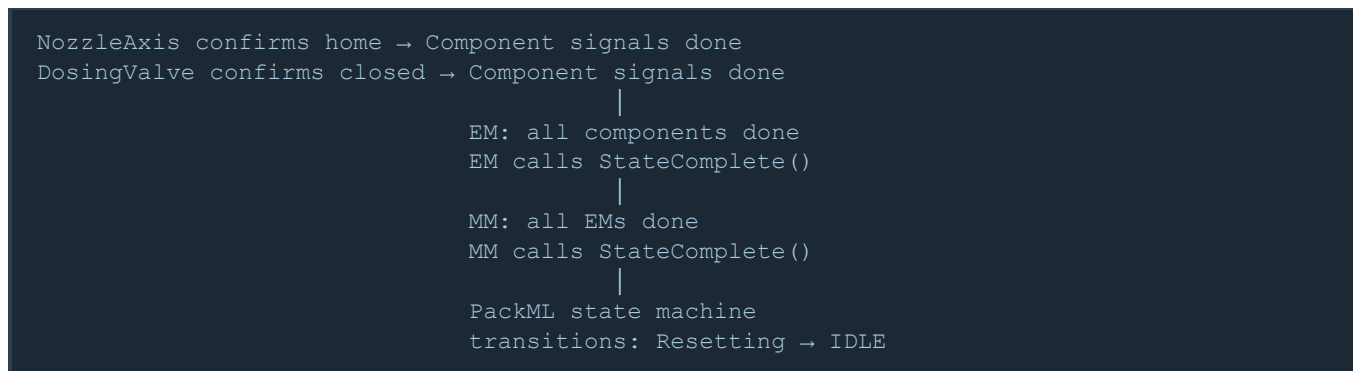
```
Operator → HMI → Control Source
                |
                ▼
Machine Module (MM)
  enters Resetting state
  calls M_Resetting() on itself
                |
```



The MM does not reach down past its direct children. Each level commands only its own direct children. The EM commands its components. A component never receives a command from the MM directly.

How Completion Propagates — Bottom Up

The reverse path is equally important:

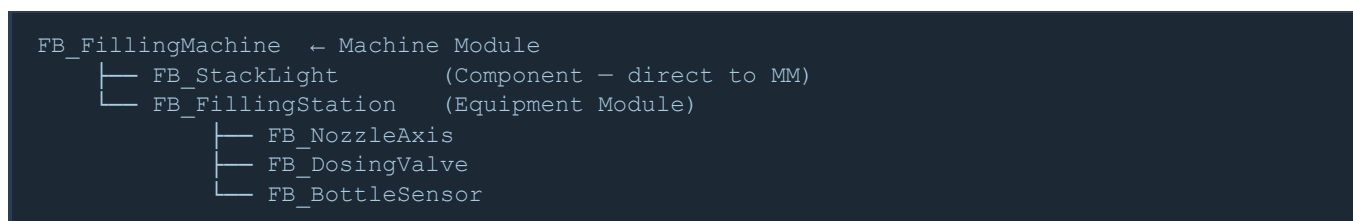


This bottom-up propagation is automatic once you register your components and submodules correctly — that registration mechanism is Article 6. The point here is the principle: no level skips a level, in either direction.

Does Every Component Need an Equipment Module?

It is important to be aware that an Equipment module is not always required, and a Machine Module can have a direct connection to a Component.

A stack light, for example, shows overall machine status. It does not belong to any particular station — it belongs to the machine itself. So it registers directly with the MM as a component, with no EM in between.



The Hierarchy Decision Rule

When you design a new machine in SPT, ask these three questions in order:

1. Is this a physical device or a logical concern? → Component.
2. Is this a group of devices that always work together as one functional unit? → Equipment Module containing those Components.
3. Does this unit coordinate other units? → Machine Module (only one per project), or a Submodule of a Submodule if genuinely warranted.

If you find yourself putting motion commands inside the Machine Module, or putting state-transition logic inside a Component — stop. The hierarchy boundary has been crossed.

Art. 3**The Four SPT Libraries**

What This Article Covers

Articles 1 and 2 established what SPT is and how the three-level hierarchy works conceptually. This article answers the practical question: when you open TwinCAT and add references to your PLC project, what exactly is in each library and what do you use it for?

The Dependency Chain

Before listing each library's contents, there is one rule to understand: the libraries have a dependency order.

```

Your Project
└─ requires: SPT_Base_Types      ← always, no exceptions
             SPT_Components     ← when using pre-built hardware FBs
             SPT_Uutilities     ← when using helper functions
             SPT_Event_Logger   ← when using structured alarms/events

SPT_Base_Types
└─ requires: Tc3_PackML_V2      ← the Beckhoff PackML library underneath

```

SPT_Base_Types is the only mandatory library. Everything else is optional and added as your machine needs it. You never reference Tc3_PackML_V2 directly in your project — SPT_Base_Types already pulls it in and wraps it.

Library 1 — SPT_Base_Types

This is the core. It contains the base classes that every Module and Component in your project will inherit from. Without it, there is no SPT.

Function Blocks in SPT_Base_Types

FB_PackML_BaseModule will be inherited by the Machine Module and all Submodules. FB_ComponentBase will be inherited by all components. These two are the most important FBs in the entire framework.

The full list:

FB	Type	Who uses it
FB_PackML_BaseModule	Abstract	Your Machine Module and all Equipment Modules — inherit this
FB_ComponentBase	Abstract	All your Components — inherit this
FB_ControlSource	Abstract	Your HMI bridge FB — inherit this
FB_CyclicFB	Abstract	Base for anything that needs a cyclic update loop — rarely used directly
FB_BaseFB	Abstract	Root of the entire inheritance chain — never used directly
FB_TraceLog	Concrete	Debug trace logging — use directly for writing trace messages

Every FB in this list is also public access to `FB_TraceLog`. You never instantiate `FB_PackML_BaseModule` directly — you write `FB_FillingStation` EXTENDS `FB_PackML_BaseModule` and instantiate your own FB.

What `FB_BaseFB` Gives Every FB

`FB_BaseFB` contains property backers for all `I_BaseFB` properties. This is the most basic building block of all framework function blocks. Because the entire inheritance chain flows through it, every FB in your project — whether a Module or a Component — automatically inherits three trace methods:

Method	What it does
<code>Trace(Message)</code>	Writes a debug string to the global trace log
<code>Marker(Message)</code>	Same, but marks it with a flag for easy filtering
<code>TraceWithJson(Message, Json)</code>	Writes a message plus a JSON payload — useful for structured logging

You call these freely from inside any Module or Component method. They are your first diagnostic tool.

What `FB_ComponentBase` Adds

`FB_ComponentBase` extends `FB_CyclicFB` and implements `I_ComponentBase`. It contains property backers for all `I_ComponentBase` properties as well as basic housekeeping code for all component-level function blocks.

It provides protected methods you override in your own component:

Method	When you override it
<code>Initialize()</code>	One-time setup — link I/O, configure parameters, initialize sub-components
<code>CyclicLogic()</code>	Every scan — update device status, run cyclic checks
<code>HMICommunication()</code>	Every scan — process HMI commands directed at this component
<code>CreateEvents()</code>	Once — declare this component's alarm events

All are PROTECTED. You call `SUPER^.Method()` inside your override so the base class housekeeping still runs. Articles 4 and 5 go into each of these in depth.

Interfaces in `SPT_Base_Types`

The library also defines the interfaces that make the registration system work (Article 6). Key ones:

Interface	Implemented by
<code>I_PackML_BaseModule</code>	<code>FB_PackML_BaseModule</code> — used when registering submodules
<code>I_ComponentBase</code>	<code>FB_ComponentBase</code> — used when registering components
<code>I_ControlSource</code>	<code>FB_ControlSource</code> — used when registering the HMI bridge
<code>I_BaseFB</code>	Root interface — all framework objects implement it

You will pass these interface references into the registration methods. You never need to implement these interfaces yourself — inheriting the correct base FB is enough.

Data Types in SPT_BaseTypes

The library defines a set of structs used as standardized HMI and status data containers:

Struct	Purpose
ST_ComponentBase_HMI	Standard HMI block every component gets — contains Command and Status sub-structs
ST_PackML_BaseModule_HMI	Standard HMI block for every module

These appear as properties on your base FB instances and become your structured HMI data contract — the HMI reads and writes these structs, not raw variables.

Library 2 — SPT_Components

This library contains ready-to-use Component FBs for common hardware types. You use them directly — either as-is by instantiating them inside your Equipment Module, or as base classes to extend.

What is Pre-Built

Category	FBs Available
Cylinders	Single-output, dual-output, dual-output with sensor feedback
Sensors	Digital input, analog input
Motion Control	Servo axis component wrapping Tc2_MC2 motion FBs
MODBUS-RTU	Serial device communication component
PackML Event	Component for raising PackML-standard events/alarms

The Key Rule for SPT_Components

Every pre-built component already inherits FB_ComponentBase. So when you extend one, you get the full base class chain automatically — Initialize(), CyclicLogic(), HMICommunication(), alarms — all already wired. You only override what is specific to your device.

Library 3 — SPT_Uilities

This library has no SPT-specific framework objects. It is a collection of small, general-purpose reusable FBs and functions that commonly appear in machine projects.

What Is Inside

Category	Examples
Scaling	Analog input scaling (raw counts → engineering units)
Buffers	FIFO and LIFO data buffers
Comparators	Deadband comparator, window comparator

Timers/Counters	Extended timer patterns not in the standard library
String Helpers	Formatting, parsing utilities

SPT_Uilities has no dependency on SPT_Base_Types. It can be used in any TwinCAT 3 project even without SPT.

Library 4 — SPT_Event_Logger

This library wraps Beckhoff's Tc3_EventLogger and tunes it for SPT. Rather than using the raw TwinCAT event logger API directly, you use this wrapper — it integrates with the alarm monitoring patterns in FB_ComponentBase and the Module hierarchy.

What It Provides

Object	Purpose
Logging functions	F_LogEvent, F_LogMessage — write structured events to the TwinCAT event log
Alarm FB integration	Works with CreateEvents() in FB_ComponentBase to register component-specific alarms
Severity levels	Info, Warning, Error — maps to PackML alarm severity standards

The important point here is that you do not call the TwinCAT event logger directly in your SPT project. All alarm and event creation goes through SPT_Event_Logger functions and the CreateEvents() / component alarm pattern. This is covered fully in Article 10.

How They Map to the Hierarchy

Connecting everything to Article 2's hierarchy:

```
Machine Module (FB_Machine)
  inherits: FB_PackML_BaseModule ← SPT_Base_Types
  —
  Equipment Module (FB_FillingStation)
    inherits: FB_PackML_BaseModule ← SPT_Base_Types
    —
    FB_NozzleAxis
      inherits: FB_ServoAxisBase ← SPT_Components
      (which extends FB_ComponentBase ← SPT_Base_Types)
    —
    FB_DosingValve
      inherits: FB_DigitalOutput ← SPT_Components
    —
    FB_PressureSensor
      inherits: FB_AnalogInput ← SPT_Components
      uses: F_ScaleAnalog ← SPT_Uilities
  —
  FB_StackLight (direct component of MM)
    inherits: FB_ComponentBase ← SPT_Base_Types
    alarms via: SPT_Event_Logger
```

Reference Addition Order in TwinCAT

When starting a new project, add references in this order under your PLC project → References:

4. Tc3_PackML_V2 — pulled in automatically by SPT_Base_Types, but pin the version explicitly
5. SPT_Base_Types — always first
6. SPT_Components — add when you start writing components
7. SPT_Uilities — add as needed
8. SPT_Event_Logger — add when you reach alarm implementation

Pinning library versions matters in SPT V4. The official guidance is to pin SPT_Base_Types to a specific version so that a library update does not silently break your project mid-development.

Art. 4

FB_PackML_BaseModule — The Module Base Class

Where This Fits in the Thread

Article 3 listed `FB_PackML_BaseModule` as the abstract FB in `SPT_Base_Types` that both your Machine Module and all Equipment Modules inherit from. This article opens it up completely — its inheritance chain, every method you will override, the lifecycle from power-on to running production, and the first concrete code showing a real Module declaration.

The Inheritance Chain

Every Module in your project is a custom FB that extends `FB_PackML_BaseModule`. That base class itself sits inside a chain:

```
FB_BaseFB
├── FB_CyclicFB
│   └── FB_PackML_BaseModule
│       ├── FB_FillingMachine    ← your Machine Module
│       └── FB_FillingStation    ← your Equipment Module
```

`FB_BaseFB` contributes the `Trace()`, `Marker()`, and `TraceWithJson()` methods (covered in Article 3). `FB_CyclicFB` contributes the `CyclicLogic()` entry point. `FB_PackML_BaseModule` adds everything PackML-related: the state machine engine, the state methods, registration, and `StateComplete()`.

You never touch `FB_BaseFB` or `FB_CyclicFB` directly. When you write `FB_FillingStation` EXTENDS `FB_PackML_BaseModule`, you get the entire chain automatically.

The Full Method Map

Here is a visual overview of `FB_PackML_BaseModule` before diving into each method — the entry point, the lifecycle methods, the full set of state methods, and how `StateComplete()` feeds the PackML state machine running invisibly inside the base class.

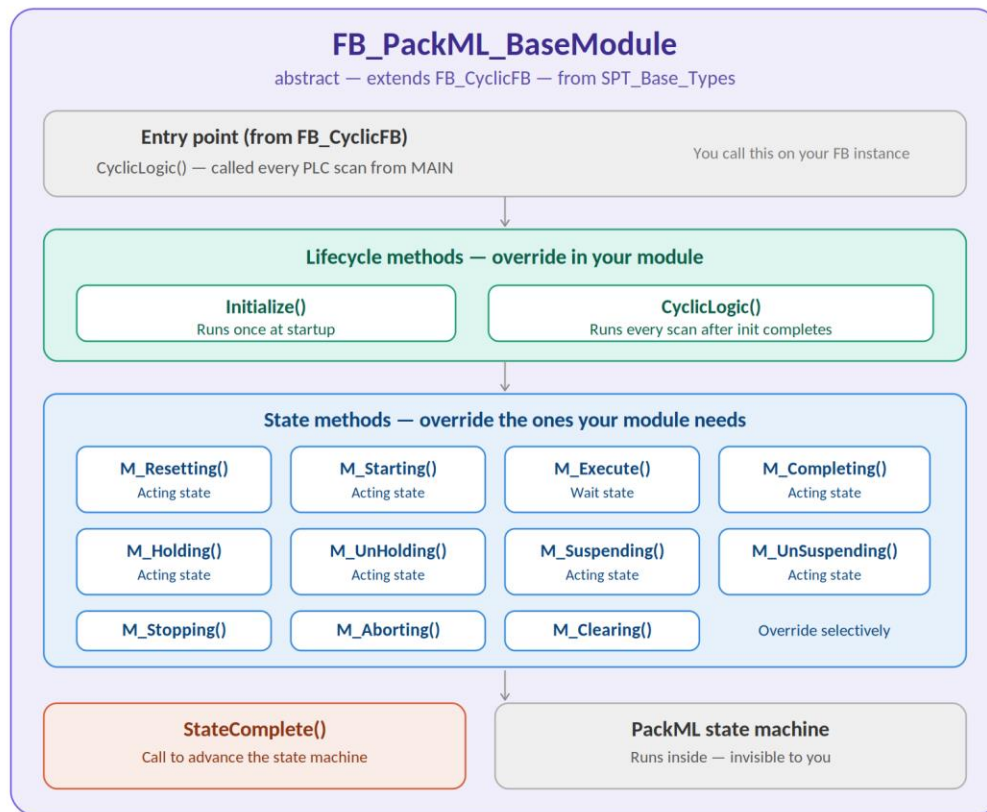


Figure 4.1 — *FB_PackML_BaseModule anatomy: inheritance chain and method categories*

The Entry Point: CyclicLogic()

CyclicLogic() is the one and only method you call on your Module instance from MAIN. It is the single entry point into the entire hierarchy every PLC scan.

```
// MAIN — this is the entire MAIN program body in an SPT project
PROGRAM MAIN
VAR
    fbMachine : FB_FillingMachine; // Your Machine Module
END_VAR

fbMachine.CyclicLogic();
```

That single call — **fbMachine.CyclicLogic()** — triggers the entire framework: initialization, state machine execution, propagation down to all Equipment Modules and Components, and propagation of completion back up. MAIN never touches any Equipment Module or Component directly.

Lifecycle Method 1: Initialize()

Initialize() is a **PROTECTED** **Public Method** that runs automatically on the first scans — before the PackML state machine becomes active. Its purpose is one-time setup: registering submodules and components (Article 6), linking I/O references, and setting parameters.

The critical rule: **Initialize()** must return **TRUE** before the framework considers the module ready. You call **SUPER^.Initialize()** at the end of your override, and return its result. The framework keeps calling **Initialize()** every scan until it returns **TRUE**.

```
// Inside FB_FillingStation EXTENDS FB_PackML_BaseModule
METHOD PROTECTED Initialize : BOOL

// Register submodules and components here (Article 6 detail)
// Link I/O references here

// Call base — returns TRUE when base init is done
Initialize := SUPER^.Initialize();
```

If your Equipment Module contains sub-components that also need to initialize, you guard the **SUPER^** call:

```
METHOD PROTECTED Initialize : BOOL

// Block until all child components have finished their own Init
IF NOT (fbNozzleAxis.InitComplete AND fbDosingValve.InitComplete) THEN
    RETURN; // Returns FALSE implicitly — framework keeps calling
END_IF

Initialize := SUPER^.Initialize();
```

This chained pattern ensures the entire hierarchy initializes from the bottom up before any state machine runs.

Lifecycle Method 2: **CyclicLogic()** — The Override

After **Initialize()** returns **TRUE**, the framework calls **CyclicLogic()** every scan. You override it in your Equipment Module to call your component FBs and run any scan-level logic.

```
// Inside FB_FillingStation
METHOD PUBLIC CyclicLogic

// Call all child component cyclic logic first
fbNozzleAxis.CyclicLogic();
fbDosingValve.CyclicLogic();
fbBottleSensor.CyclicLogic();

// Then call the base — this runs the state machine and calls
// the active state method (M_Resetting, M_Execute, etc.)
SUPER^.CyclicLogic();
```

The order here matters: `production.CyclicLogic()` calls come before `SUPER^.CyclicLogic()`. This ensures component status is up-to-date before the state method reads it. Reversing the order means your state method acts on last-scan's component status.

The State Methods

This is where your production logic lives. `FB_PackML_BaseModule` declares a `PROTECTED` virtual method for every acting state. The framework calls the appropriate method automatically when the state machine enters that state.

The Naming Convention

All state methods follow the prefix `M_`:

Method	When the framework calls it
<code>M_Resetting()</code>	State machine is in Resetting
<code>M_Starting()</code>	State machine is in Starting
<code>M_Execute()</code>	State machine is in Execute
<code>M_Completing()</code>	State machine is in Completing
<code>M_Holding()</code>	State machine is in Holding
<code>M_UnHolding()</code>	State machine is in UnHolding
<code>M_Suspending()</code>	State machine is in Suspending
<code>M_UnSuspending()</code>	State machine is in UnSuspending
<code>M_Stopping()</code>	State machine is in Stopping
<code>M_Aborting()</code>	State machine is in Aborting
<code>M_Clearing()</code>	State machine is in Clearing

You only override the methods your module actually uses. You do not need to override all of them. An Equipment Module that has no batching logic never needs `M_Completing()`.

The SUPER^ Call Rule

Every state method override must call `SUPER^.Method()`. The base class performs housekeeping — propagating the state call to registered submodules and aggregating their `StateComplete` signals. If you omit `SUPER^`, your submodules and components never receive the state notification.

```
METHOD PROTECTED M_Resetting
// Your resetting logic — issue commands to components
fbNozzleAxis.Home();
fbDosingValve.Close();

// Call base — propagates to submodules, aggregates completions
SUPER^.M_Resetting();
```

StateComplete() ^{public access} — The Transition Signal

StateComplete() is the method you call from inside a state method to signal that the current acting state has finished its work. It replaces the raw StateComplete input pin you wired manually in raw PackML.

```
METHOD PROTECTED M_Resetting

fbNozzleAxis.Home();
fbDosingValve.Close();

// Check if all work is done
IF fbNozzleAxis.AtHome AND fbDosingValve.IsClosed THEN
    THIS^.StateComplete(); // Signal: Resetting is done → advance to IDLE
END_IF

SUPER^.M_Resetting();
```

Three important rules about StateComplete():

- Rule 1 — Call it before SUPER^, or at least before the method returns. The base class reads this signal during SUPER^.M_Resetting().
- Rule 2 — Only call it from acting state methods. Calling it from M_Execute() is valid only if your machine has a defined end-of-batch cycle. For continuous production, M_Execute() never calls StateComplete() — the operator uses Stop to exit.
- Rule 3 — Do not call it from wait states (M_Idle, M_Stopped, etc.). Wait states do not have state-complete logic — they exit only via commands.

Declaring Your Modules — The First Code

Here is the complete declaration of a Machine Module and an Equipment Module for the filling station. This is what you actually write in TwinCAT.

Equipment Module

```
// FB_FillingStation.TcPOU
FUNCTION_BLOCK FB_FillingStation EXTENDS FB_PackML_BaseModule
VAR
    // Child components — declared as instances inside the EM
    fbNozzleAxis    : FB_NozzleAxis;    // Your component (Article 5)
    fbDosingValve   : FB_DosingValve;   // Your component
    fbBottleSensor  : FB_BottleSensor;  // Your component

    // Internal step variable for Execute sequence
    nExecStep       : INT;
    tonFill         : TON;
END_VAR
```

Machine Module

```
// FB_FillingMachine.TcPOU
```

```

FUNCTION_BLOCK FB_FillingMachine EXTENDS FB_PackML_BaseModule
VAR
    // Child Equipment Module
    fbFillingStation : FB_FillingStation;

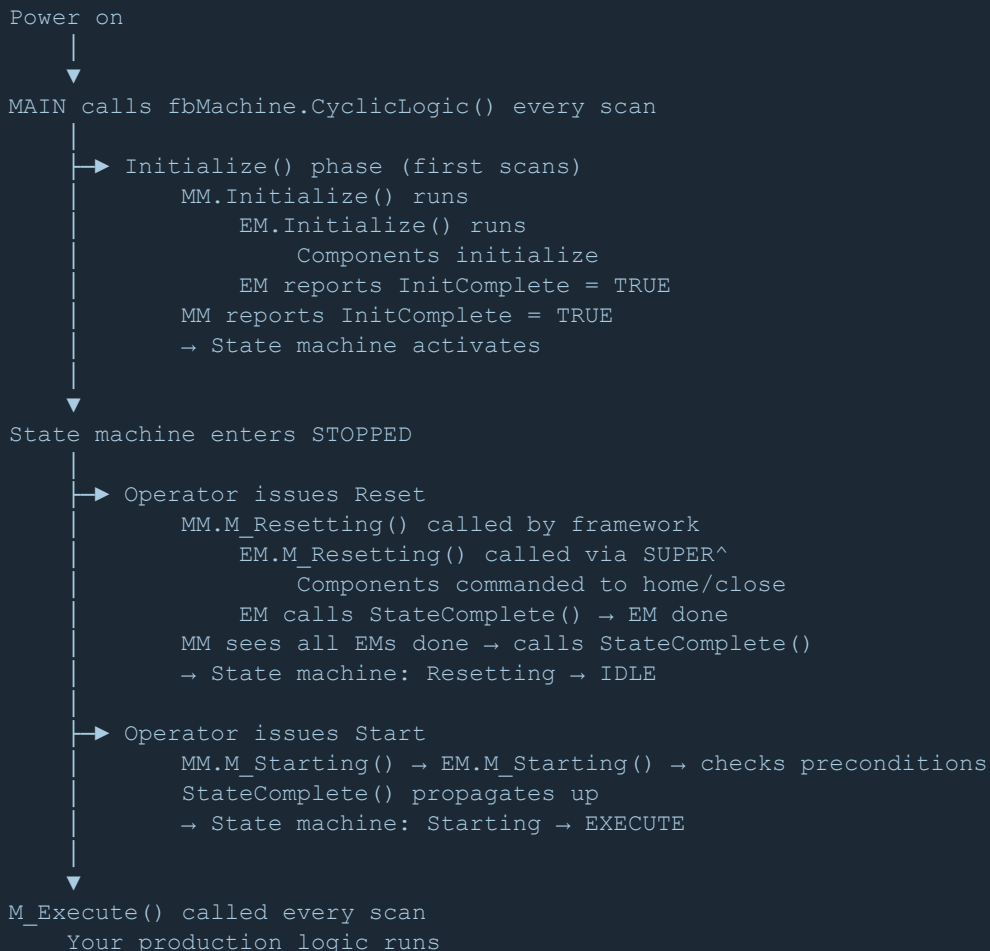
    // Stack light — direct component of the MM (no EM wrapper needed)
    fbStackLight      : FB_StackLight;
END_VAR

```

Both FBs are identical in how they are declared — EXTENDS FB_PackML_BaseModule. The difference is only what they contain and which level of the hierarchy they occupy. The Machine Module is instantiated in MAIN; the Equipment Module is instantiated inside the Machine Module.

The Lifecycle Sequence — From Power-On to Execute

The full sequence from the moment the PLC starts to the machine running production:



What the Framework Hides

Compared to raw PackML, FB_PackML_BaseModule makes three things disappear:

- Gone: the StateComplete process wiring. You no longer write `StateComplete := (ST_Resetting AND fbResetting.StateComplete) OR ...` You call `THIS^.StateComplete()` from inside the state method and the framework handles aggregation.
- Gone: the state machine FB instantiation. `PS_PackML_StateMachine_Auto` is instantiated and called inside `FB_PackML_BaseModule`. You never see it.
- Gone: output gating. You no longer write `bAxisMoveDown := ST_Execute AND fbExecute.bMoveDown`. Physical outputs are driven inside component `CyclicLogic()`, gated internally by the component's own state awareness. This moves the gate logic from MAIN into each component where it belongs.

Art. 5**FB_ComponentBase — The Component Base Class**

Where This Fits in the Thread

Article 4 covered FB_PackML_BaseModule — the base class for your Machine Module and Equipment Modules, with its M_xxx state methods and StateComplete(). This article covers the other base class named in Article 3: FB_ComponentBase — the one every Component in your project inherits from.

Key conceptual shift

Components do not know about PackML states. There is no M_Resetting() or M_Execute() in FB_ComponentBase. A component has its own small internal state machine — Idle, Busy, Error, Reset — and it is the Equipment Module's job (via M_xxx methods) to command the component appropriately for whatever PackML state is active.

The Inheritance Chain

```

FB_BaseFB
├── FB_CyclicFB
│   └── FB_ComponentBase
│       ├── FB_NozzleAxis    ← your component
│       ├── FB_DosingValve   ← your component
│       └── FB_BottleSensor   ← your component

```

Same root as Modules — FB_BaseFB gives every component Trace(), Marker(), TraceWithJson(). FB_CyclicFB gives the CyclicLogic() entry point and InitComplete property used in Article 4's chained initialization pattern. FB_ComponentBase adds component-specific housekeeping: an internal status state machine, HMI command interlocking, and alarm handling.

The Method Map

Before walking through each piece, here is the structural overview: the lifecycle methods, the HMI communication entry point, and the monitoring/alarm pair, with the resulting status outputs the parent module reads every scan.

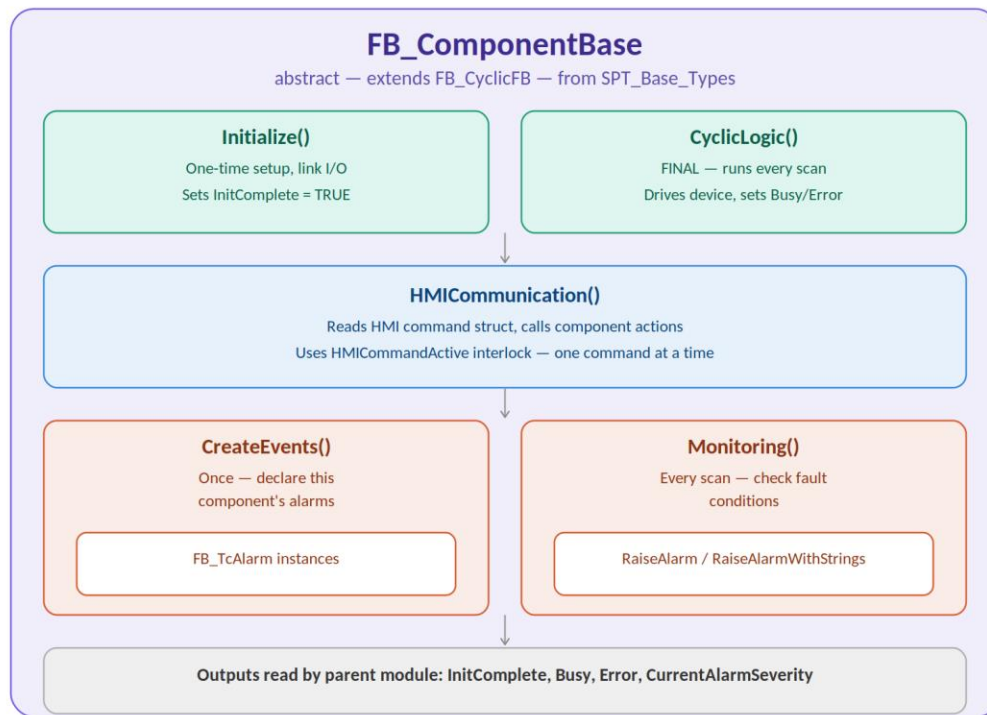


Figure 5.1 — *FB_ComponentBase anatomy: lifecycle, HMI, and monitoring*

The Internal Status Model

Every component built on **FB_ComponentBase** tracks its own internal `MainState`, separate and unaware of the PackML state machine in Article 4. The typical values are `Idle`, `Busy`, `Error`, and `Reset`. From these, the base class derives two key outputs that the parent Equipment Module reads every scan:

Output	Meaning
Busy	TRUE while <code>MainState <> Idle</code> AND <code>MainState <> Error</code> — the component is actively doing something
Error	TRUE while <code>MainState = Error</code> AND <code>MainState <> Reset</code> — the component has a fault
InitComplete	TRUE once <code>Initialize()</code> has finished its one-time setup

These three outputs are how the Equipment Module knows what a component is doing — without the component ever knowing what PackML state the machine is in. This separation is exactly the isolation principle from Article 1: the component's internal logic is independent of the production sequence around it.

Initialize() — One-Time Setup

Same pattern as Article 4's Equipment Module `Initialize()`, but at the component level it typically does three things: link any I/O references passed in, set default parameters, and create alarm objects via `CreateEvents()`.

```
// Inside FB_DosingValve EXTENDS FB_ComponentBase
METHOD PROTECTED Initialize : BOOL

// Link physical output reference (passed via FB_init or property)
// e.g. pValveOutput already set from FB_init

Initialize := SUPER^.Initialize();
```

If your component wraps sub-components — for example, an axis component that wraps both a Tc2_MC2 axis FB and a brake output FB — you use the same guarded chaining pattern shown in Article 4:

```
METHOD PROTECTED Initialize : BOOL

IF NOT (fbBrakeOutput.InitComplete) THEN
    RETURN;
END_IF

Initialize := SUPER^.Initialize();
```

CyclicLogic() — FINAL, Not Overridable the Same Way

This is the most important difference from Module CyclicLogic(). At the component level, CyclicLogic() is declared PUBLIC FINAL — you still override it, but you must call SUPER^.CyclicLogic(), and the framework expects a specific shape:

```
METHOD PUBLIC FINAL CyclicLogic

// 1. Call any sub-components' CyclicLogic first
fbBrakeOutput.CyclicLogic();

// 2. Your component's own logic / internal state machine
CASE MainState OF
    Idle:
        // waiting for a command
    Busy:
        // executing the current command
    Error:
        // fault condition active
    Reset:
        // clearing fault
END_CASE

// 3. Call base — runs HMI communication, monitoring, alarm aggregation
SUPER^.CyclicLogic();

// 4. Derive status outputs
_Busy := MainState <> Idle AND MainState <> Error;
_Error := MainState = Error AND MainState <> Reset;
```

The order is fixed: sub-components first, your logic second, SUPER^ third, status derivation last. This mirrors the Equipment Module pattern from Article 4 — children always go first so their status is current when your logic and the base class run.

HMICommunication() — Component-Level HMI Commands

This is how an operator interacts with a component directly — for example, jogging an axis or manually opening a valve from a maintenance screen. The base class provides an interlock: HMICommandActive_Base and HMICommandActive_Descendant ensure only one HMI command fires per scan, whether it's a built-in command or one you add.

```
METHOD PROTECTED HMICommunication

SUPER^.HMICommunication();

// Rising-edge trigger for momentary commands, interlocked
HMICommand_Valve_RT(
    CLK := ComponentBase_HMI.Status.HMIControlAvailable
        AND NOT HMICommandActive_Base
        AND (DosingValve_HMI.Command.Open XOR DosingValve_HMI.Command.Close)
);

// Tell the base a command is active this scan
HMICommandActive_Descendant :=
    ComponentBase_HMI.Status.HMIControlAvailable
    AND (DosingValve_HMI.Command.Open OR DosingValve_HMI.Command.Close);

IF HMICommand_Valve_RT.Q THEN
    IF DosingValve_HMI.Command.Open THEN
        OpenValve();
    ELSIF DosingValve_HMI.Command.Close THEN
        CloseValve();
    END_IF
END_IF

// Update HMI status fields every scan
DosingValve_HMI.Status.IsOpen := bValveOutput;
```

ST_ComponentBase_HMI (named in Article 3) is the struct family this pattern relies on — Command fields are written by the HMI, Status fields are written by the component. The HMI never touches the component's internal variables directly; everything passes through this struct.

Monitoring(), CreateEvents(), and Alarms

CreateEvents() runs once and declares the component's alarm objects — typically an array of FB_TcAlarm. Monitoring() runs every scan and checks fault conditions, raising or clearing those alarms.

```
// Declared in VAR section
VAR
    aAlarms : ARRAY[0..1] OF FB_TcAlarm;
```

```

END_VAR

METHOD PROTECTED CreateEvents
SUPER^.CreateEvents();
// aAlarms[0] and [1] configured via Event Logger XML (Article 10 detail)

```

```

METHOD PROTECTED Monitoring
SUPER^.Monitoring();

// Example: valve commanded open but feedback never confirms
IF bCmdOpen AND NOT bValveOpenFeedback AND tonValveTimeout.Q THEN
    IF NOT aAlarms[0].bRaised THEN
        RaiseAlarmWithStrings(Alarm := aAlarms[0], String_1 := 'Open', String_2 := '');
    END_IF
ELSIF aAlarms[0].bRaised AND bValveOpenFeedback THEN
    aAlarms[0].Clear(0, 0);
END_IF

```

RaiseAlarm() and RaiseAlarmWithStrings() are the two methods you call — the difference is whether the alarm message needs contextual text inserted (like which command timed out). Both automatically include the component's Name as part of the alarm identity, so the same alarm definition reused across multiple instances of a component (e.g. four identical cylinders) reports which instance raised it. Article 10 covers the full alarm configuration; here you only need to know these two methods exist and where they're called from.

Complete Worked Example — FB_DosingValve

Putting it together — a full, minimal, working component:

```

FUNCTION_BLOCK FB_DosingValve EXTENDS FB_ComponentBase
VAR_INPUT
    bValveOpenFeedback : BOOL;    // sensor confirming valve is open
END_VAR
VAR_OUTPUT
    bValveOutput        : BOOL;    // physical digital output
END_VAR
VAR
    MainState          : E_DosingValveState := E_DosingValveState.Idle;
    bCmdOpen           : BOOL;
    bCmdClose          : BOOL;
    tonTimeout         : TON;
    aAlarms             : ARRAY[0..0] OF FB_TcAlarm;

    HMICommand_Valve_RT : R_TRIG;
    DosingValve_HMI      : ST_DosingValve_HMI;
END_VAR

```

```

// E_DosingValveState — your own enum
TYPE E_DosingValveState :
(
    Idle, Opening, Open, Closing, Closed, Error, Reset

```

```
);
END_TYPE
```

```
METHOD PROTECTED Initialize : BOOL
Initialize := SUPER^.Initialize();
```

```
METHOD PUBLIC FINAL CyclicLogic

tonTimeout(IN := (MainState = Opening OR MainState = Closing), PT := T#2S);

CASE MainState OF

    Idle, Closed:
        bValveOutput := FALSE;
        IF bCmdOpen THEN MainState := Opening; END_IF

    Opening:
        bValveOutput := TRUE;
        IF bValveOpenFeedback THEN
            MainState := Open;
        ELSIF tonTimeout.Q THEN
            MainState := Error;
        END_IF

    Open:
        bValveOutput := TRUE;
        IF bCmdClose THEN MainState := Closing; END_IF

    Closing:
        bValveOutput := FALSE;
        IF NOT bValveOpenFeedback THEN
            MainState := Closed;
        ELSIF tonTimeout.Q THEN
            MainState := Error;
        END_IF

    Error:
        bValveOutput := FALSE;

    Reset:
        bValveOutput := FALSE;
        MainState := Idle;
END_CASE

SUPER^.CyclicLogic();

_Busy := MainState <> Idle AND MainState <> Error AND MainState <> Closed;
_Error := MainState = Error;
```

```
METHOD PROTECTED Monitoring
SUPER^.Monitoring();
```

```

IF MainState = E_DosingValveState.Error AND NOT aAlarms[0].bRaised THEN
    RaiseAlarmWithStrings(Alarm := aAlarms[0], String_1 := 'Open/Close', String_2 := '');
ELSIF MainState <> E_DosingValveState.Error AND aAlarms[0].bRaised THEN
    aAlarms[0].Clear(0, 0);
END_IF

```

```

// Public methods — called BY the Equipment Module's M_xxx methods
METHOD PUBLIC Open
bCmdOpen := TRUE;
bCmdClose := FALSE;

METHOD PUBLIC Close
bCmdOpen := FALSE;
bCmdClose := TRUE;

// Public property — read BY the Equipment Module
PROPERTY PUBLIC IsOpen : BOOL
IsOpen := (MainState = E_DosingValveState.Open);

```

How the Equipment Module Uses This Component

Connecting straight back to Article 4's M_xxx methods — here is FB_FillingStation.M_Resetting() using FB_DosingValve:

```

METHOD PROTECTED M_Resetting

fbDosingValve.Close();           // command the component
fbNozzleAxis.Home();

IF fbDosingValve.IsOpen = FALSE AND fbNozzleAxis.AtHome THEN
    THIS^.StateComplete(); // Signal: Resetting is done → advance to IDLE
END_IF

SUPER^.M_Resetting();

```

Notice the clean separation: FB_FillingStation calls Close() and reads IsOpen — public methods and properties. It never touches bValveOutput, MainState, or tonTimeout. Those are private to the component. This is the isolation boundary from Article 1's Problem 2 — fixing the valve's internal timeout logic can never break the Resetting sequence, because the Equipment Module only sees Open(), Close(), and IsOpen.

Why This Makes the Component Portable

Look at FB_DosingValve again — nothing in its CyclicLogic(), Monitoring(), Open(), or Close() references FB_FillingStation, the PackML state machine, or any M_xxx method. The only framework dependency is EXTENDS FB_ComponentBase itself. You could drop this exact FB into a completely different machine,

instantiate it, call `CyclicLogicAccess` in a test program, and it would behave identically. This is what the modularity and flexibility from Article 1 means in concrete code.

Summary Table — Module vs. Component

Aspect	FB_PackML_BaseModule (Article 4)	FB_ComponentBase (this article)
Knows about PackML states	Yes — M_xxx methods	No — has its own MainState
Signals completion via	StateComplete()	Busy/Error/InitComplete outputs
CyclicLogic() override	Free-form, calls children then SUPER^	FINAL, fixed shape: children → logic → SUPER^ → status
HMI entry point	Control Source (Article 9)	HMICommunication()
Alarms	Aggregated from children	CreateEvents() + Monitoring() + RaiseAlarm
Reusable across projects	No — tied to this machine's hierarchy	Yes — by design

Art. 6**Registering Submodules and Components**

Where This Fits in the Thread

Articles 4 and 5 described how `FB_PackML_BaseModule` propagates state calls top-down and aggregates `StateComplete()` bottom-up automatically. Article 5's `FB_FillingStation.M_Resetting()` called `SUPER^.M_Resetting()` and relied on this propagation to reach `FB_DosingValve` and `FB_NozzleAxis`. This article explains the mechanism that makes that propagation work: registration.

Without registration, `SUPER^.M_Resetting()` has no knowledge of your child Submodules or Components — they are just VAR declarations inside your FB. Registration is what tells the framework they exist and establishes the parent-child link.

The Core Concept

Within each Machine and Submodule there will exist the declaration of the SubModules and Components of that Module. Each of these instance variables will be passed into the methods for `RegisterSubmodule` or `RegisterComponent`.

There are exactly two registration methods, both called from inside `Initialize()`:

Method	Used for	Interface required
<code>RegisterSubmodule()</code>	Registering a child Submodule (Equipment Module)	<code>I_PackML_BaseModule</code>
<code>RegisterComponent()</code>	Registering a child Component	<code>I_ComponentBase</code>

When creating components, the same process is used, the only difference is that the components extend `FB_ComponentBase` which implements `I_ComponentBase` and the method is `RegisterComponent`.

Both methods accept the instance by interface reference — not by direct FB reference. Because `FB_PackML_BaseModule` implements `I_PackML_BaseModule` and `FB_ComponentBase` implements `I_ComponentBase`, your custom FBs satisfy these interfaces automatically through inheritance. You pass `fbFillingStation` and the framework receives an `I_PackML_BaseModule`. No casting required.

What Registration Does Internally

When you call `RegisterSubmodule(fbFillingStation)` inside `FB_FillingMachine.Initialize()`, the framework:

9. Adds `fbFillingStation` to an internal dynamic array of `I_PackML_BaseModule` references
10. Sets the parent reference inside `fbFillingStation` back to `FB_FillingMachine` — the two-way link

`RegisterSubmodule()` gives the option to add Submodules to the internal array one at a time. This can improve readability when the number of Submodules grows large. The same applies to `RegisterComponent()`.

After this, every time `FB_FillingMachine`'s `SUPER^.M_Resetting()` runs, the framework iterates over the registered Submodule array and calls `M_Resetting()` on each one. Those Submodules do the same for their

registered Components. This is the automatic top-down propagation from Article 4 — and it only works because of this registration.

V4 Registration Pattern

Prior to v3.2.2, child Submodules were added to Submodules by iterating through a pre-initialized array of `I_PackML_BaseModule` and adding them to another internal array. This was typically the first step in the Submodule initialization process. `RegisterSubmodule()` gives the option to add Submodules to the internal array one at a time.

In V4 (which this series uses throughout), you call `RegisterSubmodule()` and `RegisterComponent()` individually — one call per child — inside your `Initialize()` override. This is cleaner and scales naturally as the machine grows.

Registration in the Machine Module

`FB_FillingMachine` has one Submodule (`FB_FillingStation`) and one direct Component (`FB_StackLight`). Both are registered in `FB_FillingMachine.Initialize()`:

```
// FB_FillingMachine EXTENDS FB_PackML_BaseModule
VAR
    fbFillingStation : FB_FillingStation;    // Submodule
    fbStackLight      : FB_StackLight;        // Direct component of the MM
END_VAR
```

```
METHOD PROTECTED Initialize : BOOL

// Register the Equipment Module as a Submodule
RegisterSubmodule(Submodule := fbFillingStation);

// Register the direct Component
RegisterComponent(Component := fbStackLight);

// Call base - returns TRUE when framework init is complete
Initialize := SUPER^.Initialize();
```

Registration in the Equipment Module

`FB_FillingStation` contains three Components. All three are registered in `FB_FillingStation.Initialize()`:

```
// FB_FillingStation EXTENDS FB_PackML_BaseModule
VAR
    fbNozzleAxis      : FB_NozzleAxis;
    fbDosingValve      : FB_DosingValve;
    fbBottleSensor     : FB_BottleSensor;
END_VAR
```

```
METHOD PROTECTED Initialize : BOOL
```

```
// Register each component — one call each
RegisterComponent(Component := fbNozzleAxis);
RegisterComponent(Component := fbDosingValve);
RegisterComponent(Component := fbBottleSensor);

// Guard: wait until all components have finished Initialize()
IF NOT (fbNozzleAxis.InitComplete
  AND fbDosingValve.InitComplete
  AND fbBottleSensor.InitComplete) THEN
  RETURN; // Returns FALSE — framework keeps calling Initialize() each scan
END_IF

Initialize := SUPER^.Initialize();
```

Two things happen here: registration and the initialization guard. These are separate concerns but both belong in `Initialize()`.

Registration can be called on the first scan — `RegisterSubmodule()` and `RegisterComponent()` are safe to call repeatedly; the framework ignores duplicate registrations. It is common practice to place registration calls unconditionally at the top of `Initialize()` before any guard.

The guard (`IF NOT InitComplete...`) is what prevents the framework from advancing past `Initialize()` before children are ready, exactly as shown in Article 5's chained initialization pattern.

The Propagation Path After Registration

With the filling station fully registered, the complete call chain for a Reset command looks like this:

```
MAIN
  fbMachine.CyclicLogic()
  |
  |— FB_FillingMachine.CyclicLogic()
  |   calls SUPER^.CyclicLogic()
  |   framework iterates registered Submodules:
  |       FB_FillingStation.CyclicLogic()
  |         calls its components' CyclicLogic() first
  |         fbNozzleAxis.CyclicLogic()
  |         fbDosingValve.CyclicLogic()
  |         fbBottleSensor.CyclicLogic()
  |       calls SUPER^.CyclicLogic()
  |         framework calls M_Resetting() on FB_FillingStation
  |         your logic: fbDosingValve.Close(), fbNozzleAxis.Home()
  |         IF done → THIS^.StateComplete()
  |         SUPER^.M_Resetting()
  |       FB_StackLight.CyclicLogic() ← direct component of MM
  |
  |— StateComplete() propagation
  |   FB_FillingStation signals done → FB_FillingMachine sees it
  |   FB_FillingMachine calls THIS^.StateComplete()
  |   PackML state machine: Resetting → IDLE
```

Every arrow in this chain flows automatically from a single `fbMachine.CyclicLogic()` call in MAIN. The registration in `Initialize()` is what built the internal arrays that drive this iteration.

The Name Property — Set During Registration

Every registered Submodule and Component carries a Name property — a string identifier used in alarm messages (Article 10) and trace logs (Articles 4 and 5). You set it inside Initialize() before or after registration:

```
METHOD PROTECTED Initialize : BOOL

// Set identity before registering
fbNozzleAxis.Name    := 'NozzleAxis';
fbDosingValve.Name   := 'DosingValve';
fbBottleSensor.Name  := 'BottleSensor';

RegisterComponent(Component := fbNozzleAxis);
RegisterComponent(Component := fbDosingValve);
RegisterComponent(Component := fbBottleSensor);

IF NOT (fbNozzleAxis.InitComplete
      AND fbDosingValve.InitComplete
      AND fbBottleSensor.InitComplete) THEN
  RETURN;
END_IF

Initialize := SUPER^.Initialize();
```

When FB_DosingValve later calls RaiseAlarmWithStrings(), the framework automatically prepends 'DosingValve' as the first string argument in the alarm message — no extra code needed. This was noted in Article 5 and now you can see where the Name value originates.

What Happens If You Forget to Register

This is the most common mistake in SPT and the most confusing to debug:

Forgotten registration	Symptom
RegisterSubmodule() not called for an EM	M_Resetting() and all other state methods are never called on that EM — it silently does nothing during state transitions
RegisterComponent() not called for a Component	Component CyclicLogic() is never called by the framework — the component never runs
InitComplete guard missing	The state machine activates before components are ready — first Resetting attempt may fail with undefined outputs
Name not set	Alarm messages and trace logs show an empty string for the component identity — difficult to diagnose which instance raised an alarm

None of these produce a compile error. They are runtime logic gaps — which is why the registration pattern must become a habit applied to every new child the moment it is declared in VAR.

Complete Registration Pattern — Rule Summary

Applied to the filling station hierarchy from Articles 4 and 5:

```
FB_FillingMachine.Initialize()
  RegisterSubmodule(fbFillingStation)      ← Submodule
  RegisterComponent(fbStackLight)          ← direct Component of MM
  [no guard needed — no sub-init dependency at MM level in this example]
  Initialize := SUPER^.Initialize()

FB_FillingStation.Initialize()
  fbNozzleAxis.Name      := 'NozzleAxis'
  fbDosingValve.Name     := 'DosingValve'
  fbBottleSensor.Name    := 'BottleSensor'
  RegisterComponent(fbNozzleAxis)
  RegisterComponent(fbDosingValve)
  RegisterComponent(fbBottleSensor)
  IF NOT all InitComplete → RETURN
  Initialize := SUPER^.Initialize()

FB_NozzleAxis.Initialize()      ← Component (Article 5 pattern)
  Initialize := SUPER^.Initialize()

FB_DosingValve.Initialize()      ← Component (Article 5 pattern)
  Initialize := SUPER^.Initialize()

FB_BottleSensor.Initialize()     ← Component (Article 5 pattern)
  Initialize := SUPER^.Initialize()
```

This pattern — register, name, guard, SUPER^ — applied consistently at every level, is all that is needed to make the automatic propagation described across Articles 4, 5, and 6 work correctly.

Art. 7**Initialize() and CyclicLogic() in Depth**

Where This Fits in the Thread

Articles 4, 5, and 6 introduced Initialize() and CyclicLogic() in the context of each base class and the registration pattern. You know that Initialize() runs once and CyclicLogic() runs every scan, and you know the SUPER^ call rules. This article goes one level deeper: what actually happens inside the framework across these two methods scan by scan, what the internal _InitComplete gate does, how I/O references are correctly linked in Initialize(), and the precise execution order that CyclicLogic() must preserve throughout the entire hierarchy.

The Internal _InitComplete Gate

Because an Initialize() method will take multiple scans to complete, the method is continually called until it returns TRUE at the end of the CASE statement. This program design also requires us to first call the Initialize() method at the beginning of any CyclicLogic() method if it has not been completed. The variables and methods required for this are already declared in the base classes.

The mechanism that enforces this is built into FB_CyclicFB — the common ancestor of both FB_PackML_BaseModule and FB_ComponentBase. Its internal pattern is:

```
// Inside FB_CyclicFB — framework code, not yours
METHOD PUBLIC CyclicLogic

IF NOT _InitComplete THEN
    _InitComplete := Initialize();
    RETURN;      // exits immediately — no production logic runs this scan
END_IF

SUPER^.CyclicLogic();
```

_InitComplete is a private BOOL declared inside FB_CyclicFB. The public property InitComplete gives read access to it — this is what the parent module checks in Article 6's initialization guard (IF NOT fbDosingValve.InitComplete THEN RETURN).

Once _InitComplete is TRUE for a given FB, the gate never re-evaluates. Initialize() is never called again for that instance within the same PLC session. A warm restart resets _InitComplete to FALSE — the full initialization sequence reruns on the next scan.

How Many Scans Does Initialize() Take?

This depends entirely on your implementation. The minimum is one scan — an Initialize() that has nothing to wait for returns SUPER^.Initialize() immediately, which returns TRUE on its first call.

In practice, multi-scan initialization occurs when:

- A child Component's hardware requires time to respond (e.g. a drive enabling)
- You use Initialize() to trigger a one-shot configuration write and wait for acknowledgement

- The chained `publicAccess` 6 waits on multiple children

The framework simply keeps calling `Initialize()` each scan and returning early from `CyclicLogic()` until the whole chain resolves. There is no timeout built in — if a child never sets `InitComplete := TRUE`, the machine never starts. This is intentional: a partially initialized machine must not run.

What to Put Inside `Initialize()` — and What Not To

`Initialize()` is not a one-time `M_Resetting()`. It runs before the PackML state machine exists. Its sole purpose is structural setup — establishing the connections the framework needs to operate.

Always goes in `Initialize()`

```
METHOD PROTECTED Initialize : BOOL

// 1. Register children (Article 6)
RegisterSubmodule(Submodule := fbFillingStation);
RegisterComponent(Component := fbStackLight);

// 2. Set child Names (Article 6)
fbFillingStation.Name := 'FillingStation';
fbStackLight.Name     := 'StackLight';

// 3. Link I/O references — pointer or REFERENCE TO assignment
fbDosingValve.SetOutputRef(ADR(GVL.bValveOutput));
fbNozzleAxis.SetAxisRef(GVL.AxisRef_Nozzle);

// 4. Set fixed parameters that never change at runtime
fbDosingValve.OpenTimeout := T#2S;
fbNozzleAxis.HomeVelocity := 50.0;

Initialize := SUPER^.Initialize();
```

Never goes in `Initialize()`

- Motion commands (`MC_Home`, `MC_MoveAbsolute`) — these belong in `M_Resetting()` or `M_Starting()`
- Conditional production logic — that belongs in `M_Execute()`
- Anything that should repeat after a fault and reset — `Initialize()` only runs once

The distinction is: `Initialize()` wires the structure; `M_xxx` methods run the machine.

I/O Linking Patterns

The most common `Initialize()` task after registration is linking physical I/O to components. SPT components never hold `%I*` or `%Q*` variables directly — I/O addresses are mapped in the GVL and passed into components as references or pointers during `Initialize()`.

There are two patterns in practice:

Pattern 1 — REFERENCE TO via a setter method

The component exposes a `public Ref` method or property that accepts REFERENCE TO BOOL (or the relevant type). The Submodule calls it in `Initialize()`:

```
// Inside FB_DosingValve
METHOD PUBLIC SetOutputRef
VAR_INPUT
    Ref : REFERENCE TO BOOL;
END_VAR

pValveOutput REF= Ref;
```

```
// Inside FB_FillingStation.Initialize()
fbDosingValve.SetOutputRef(Ref := GVL.bValveOutput);
```

The component then writes to `pValveOutput` in its `CyclicLogic()`. No %Q* variable lives inside the component FB itself — the physical address stays in the GVL.

Pattern 2 — ADR() with POINTER TO

When the component uses a pointer internally (common for axis references):

```
// Inside FB_FillingStation.Initialize()
fbNozzleAxis.pAxisRef := ADR(GVL.AxisRef_Nozzle);
```

Both patterns achieve the same result: the component operates on real hardware I/O without owning the I/O declaration. This is what keeps components portable (Article 5) — you pass a different reference for a different machine and the component adapts.

CyclicLogic() — The Execution Order Contract

It is important for all `CyclicLogic()` methods to be called cyclically. While this might seem obvious, there will be times when you might want to alter this, and it is important not to do so. If you have code that is not needed cyclically then call it conditionally from the `CyclicLogic()` method, but always call the `CyclicLogic()` methods. This will ensure that the internal hierarchy of `CyclicLogic()` methods and the state machine is never broken.

This is one of the firmest rules in SPT. Conditional skipping — IF `bSomethingActive` THEN `fbDosingValve.CyclicLogic()`; END_IF — breaks alarm monitoring, HMI communication, and the `InitComplete` gate for that component. Every instance's `CyclicLogic()` runs every scan, unconditionally.

The ordering rule inside any `CyclicLogic()` override is fixed and consistent across the whole hierarchy:

Step	Action	Why
1	Children first	Ensures their status is current before your logic reads it
2	Your logic	Reads up-to-date child status, runs your code
3	SUPER^.CyclicLogic()	Framework runs state machine, calls M_xxx, aggregates completion

4	Status derivation (Components only)	_Busy, _Error updated after SUPER^
---	-------------------------------------	------------------------------------

The diagram below shows this order across the full hierarchy for one scan, from the single MAIN call down through the Machine Module, the Equipment Module, its Components, and back up through the SUPER^ calls that trigger the active state method and aggregate completion.

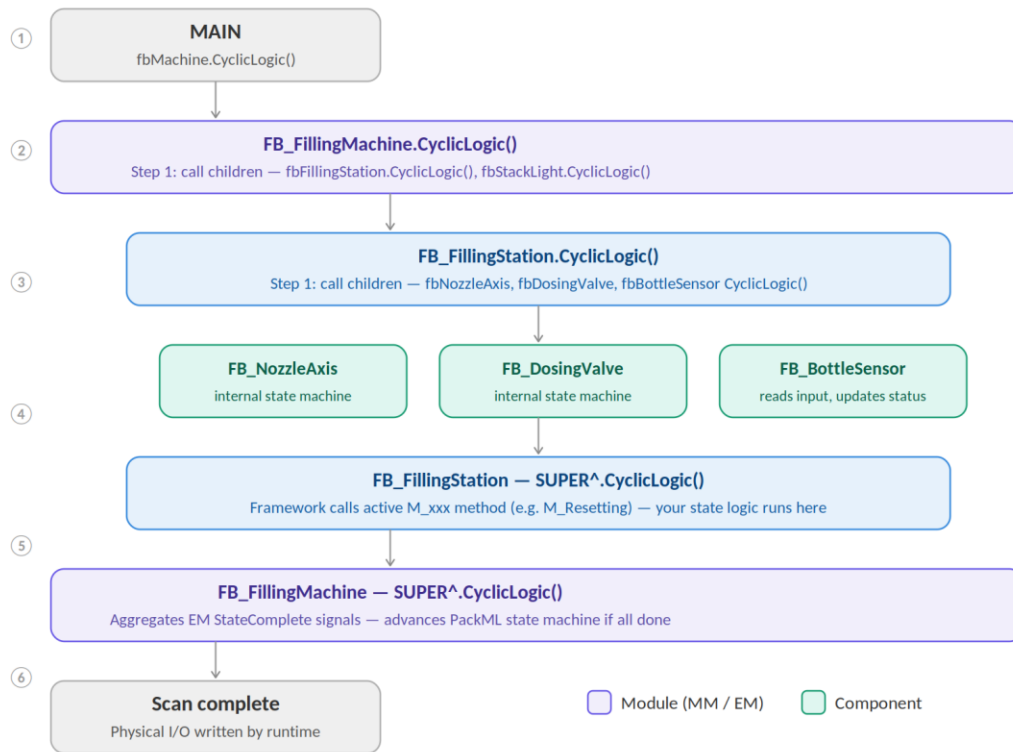


Figure 7.1 — CyclicLogic execution order across the SPT hierarchy in one PLC scan

Breaking the Order — What Goes Wrong

Inverting the call order is the most dangerous mistake in SPT, and it produces subtle bugs rather than errors:

```
// WRONG — SUPER^ called before children
METHOD PUBLIC FINAL CyclicLogic

SUPER^.CyclicLogic(); // M_Resetting() reads stale child status
fbNozzleAxis.CyclicLogic(); // too late — state method already evaluated
fbDosingValve.CyclicLogic();
```

The consequence: `M_Resetting()` evaluates `fbNozzleAxis.AtHome` based on last scan's status. The axis reaches home on scan N, sets its status on scan N, but `M_Resetting()` doesn't see it until scan N+1. In most cases this

causes a one-scan delay **public hooks** — but in fast state machines or safety-related checks, it is a real timing error.

Conditionally skipping a child's `CyclicLogic()` is worse:

```
// WRONG — conditional call
IF fbAutoSM.ST_Execute THEN
    fbDosingValve.CyclicLogic();
END_IF
```

`FB_DosingValve` now stops monitoring alarms, stops processing HMI commands, and potentially stalls its own internal state machine whenever the machine is not in `Execute`. The `Monitoring()` method inside `FB_ComponentBase` — called by `SUPER^.CyclicLogic()` — never runs. An active alarm is never cleared. This violates the framework contract and produces faults that are very hard to trace back to a skipped `CyclicLogic()` call.

MAIN — The Entry Point Options

The first call to `CyclicLogic()` of the Machine Module can be handled at a couple of different levels. Some prefer to add a `CyclicLogic()` method onto the `MAIN` POU, which is acceptable, but then you must also add the `Initialize()` method to `MAIN`. The other option is to declare `Machine : FB_Machine` within `MAIN` (assuming that `FB_Machine` `EXTENDS` `FB_PackMLBaseModule`) and then letting `MAIN` call `Machine.CyclicLogic()`.

Option A — Simple (recommended for most projects)

```
// MAIN
PROGRAM MAIN
VAR
    fbMachine      : FB_FillingMachine;
    fbControlSource : FB_FillingControlSource; // Article 9
END_VAR

fbMachine.CyclicLogic();
fbControlSource.CyclicLogic();
```

`MAIN` has no `Initialize()`, no logic — it is a pure dispatcher. `FB_FillingMachine.CyclicLogic()` handles its own initialization gate internally. This is the cleanest structure and the one recommended when starting out.

Option B — MAIN with its own Initialize()

While this is the simplest way to get started there can be some benefits to `MAIN` having its own `Initialize()` method that runs once before the remaining code is initialized and run.

This pattern is used when you need something to happen at PLC start before the SPT hierarchy initializes — for example, reading a recipe from a file, connecting to a database, or applying a configuration that the machine hierarchy depends on during its own `Initialize()`.

```
// MAIN with its own Init flag
PROGRAM MAIN
```

```

VAR
    fbMachine      : FB_FillingMachine;
    fbControlSource : FB_FillingControlSource;
    bMainInitDone   : BOOL := FALSE;
    fbRecipeLoader  : FB_RecipeLoader;    // custom, outside SPT hierarchy
END_VAR

IF NOT bMainInitDone THEN
    // One-time pre-init: load recipe, configure network, etc.
    fbRecipeLoader.CyclicLogic();
    IF fbRecipeLoader.Done THEN
        bMainInitDone := TRUE;
    END_IF
    RETURN;    // SPT hierarchy does not run until MAIN init is done
END_IF

fbMachine.CyclicLogic();
fbControlSource.CyclicLogic();

```

Notice the pattern is identical to the framework's own `_InitComplete` gate — you are extending it one level above the framework.

The Startup Sequence — First Five Scans

Combining everything across Articles 4–7, the complete scan-by-scan startup for the filling station:

```

Scan 1
    fbMachine.CyclicLogic() called
    _InitComplete = FALSE → calls Initialize()
    RegisterSubmodule(fbFillingStation)
    RegisterComponent(fbStackLight)
    calls fbFillingStation.CyclicLogic() internally
    _InitComplete = FALSE → calls Initialize()
        fbNozzleAxis.Name := 'NozzleAxis'
        RegisterComponent(fbNozzleAxis)
        fbDosingValve.Name := 'DosingValve'
        RegisterComponent(fbDosingValve)
        fbBottleSensor.Name := 'BottleSensor'
        RegisterComponent(fbBottleSensor)
        fbDosingValve.SetOutputRef(GVL.bValveOutput)
        fbNozzleAxis.pAxisRef := ADR(GVL.AxisRef_Nozzle)
        Guard: NOT fbNozzleAxis.InitComplete → RETURN (FALSE)
    Initialize() returns FALSE → _InitComplete stays FALSE

Scan 2
    Components run their own Initialize() internally
    Each returns TRUE on this scan (simple components)
    Guard in FB_FillingStation.Initialize() now passes
    SUPER^.Initialize() called → returns TRUE
    FB_FillingStation._InitComplete = TRUE

Scan 3
    FB_FillingMachine.Initialize() sees fbFillingStation.InitComplete = TRUE
    SUPER^.Initialize() called → returns TRUE
    FB_FillingMachine._InitComplete = TRUE

```

```
Scan 4 onwards
  _InitComplete = TRUE everywhere
  Normal CyclicLogic() runs — PackML state machine enters STOPPED
  Machine waits for Reset command
```

The exact number of scans varies — more children, more complex initialization patterns, more scans. But the shape is always the same: bottom-up initialization, one level per scan group, then normal operation begins.

[public access](#)

To Be Continued

public access

This white paper currently covers Articles 1 through 7. The following articles are in development and will be released in subsequent editions:

Article	Title	What It Covers
8	Overriding State Methods	M_Resetting, M_Execute, M_Stopping in depth; NoStateTasksToComplete; worked examples at both MM and EM levels
9	Control Source	I_ControlSource interface; writing your own HMI bridge; registering it with the Machine Module
10	Alarms and Events	FB_TcAlarm; AddAlarm; component alarms vs module alarms; RaiseAlarm patterns; reset strategies
11	Machine Code Progression	Recommended build order: skeleton first, then state by state, then components; the SPT development workflow
12	Putting It All Together	Complete filling station implementation in SPT from scratch — all layers, all states, all components

A Note on This Paper

This white paper is free and addressed to all industrial software engineers working with or considering the SPT framework. It may be shared, printed, reproduced, or referenced in any context without restriction. The only goal is to lower the barrier to entry for a framework that genuinely improves the quality, maintainability, and portability of machine control software.

Questions, corrections, and feedback are welcome via [LinkedIn](#).